Augmented Mechanism Design: Protective Extensions for Pure Economic Mechanisms

Faraday1 & JARVIS *VibeSwap Protocol — vibeswap.io March 2026*

Abstract

Pure economic mechanisms — bonding curves, Harberger taxes, continuous double auctions — are mathematically elegant but socially vulnerable. Each encodes a powerful allocative principle, yet each contains a structural weakness that sophisticated actors can exploit. We identify a universal pattern we call **Augmented Mechanism Design (AMD)**: the systematic addition of protective extensions to pure mechanisms that preserve their core incentive properties while shielding legitimate participants from adversarial exploitation. We formalize the augmentation operator **A(M) → M'**, demonstrate it across three case studies — Augmented Bonding Curves (ABC), Augmented Harberger Taxes (AHT), and Commit-Reveal Batch Auctions (CRBA) — and argue that augmentation, not replacement, is the correct response to mechanism failure. Each case study is grounded in deployed or deployment-ready Solidity implementations. We further propose that the CKB/Nervos UTXO cell model provides a natural computational substrate for augmented mechanisms due to its explicit state ownership and composable verification semantics.

Keywords: mechanism design, Harberger tax, bonding curves, MEV, commit-reveal, augmentation, cooperative capitalism, UTXO, CKB

1. Introduction

1.1 The Problem with Pure Mechanisms

Mechanism design — the "engineering" side of game theory (Hurwicz, 1960; Myerson, 1981) — constructs rules of interaction to achieve desired social outcomes given that agents act in self-interest. The field has produced powerful primitives: Vickrey auctions, bonding curves, Harberger taxes, automated market makers. Each is elegant in isolation. Each breaks under adversarial pressure.

The failure mode is consistent: pure mechanisms assume a population of agents that is either uniform or honestly heterogeneous. In practice, populations contain **adversarial specialists** — MEV bots, domain barons, early-dumpers, sandwich attackers — who exploit the gap between mechanism theory and mechanism reality.

The conventional response is replacement: if Harberger taxes enable plutocratic takeover, abandon them. If bonding curves enable pump-and-dump, prohibit them. If continuous trading enables MEV, accept it as cost.

We propose an alternative: **augmentation**.

1.2 The Augmentation Thesis

> ***Thesis****: For any pure mechanism M with known vulnerability V, there exists an augmentation function A such that M' = A(M) preserves the core allocative property of M while neutralizing V, without introducing vulnerabilities of equal or greater severity.*

This is not a universal theorem — it is a design methodology. The claim is that mechanism failure is almost never intrinsic to the mechanism's core principle, but rather a consequence of insufficient protective structure around that principle. The fix is not to discard the principle but to armor it.

1.3 Contributions

1. **Formal framework** for augmented mechanism design (Section 2)
2. **Three case studies** demonstrating the pattern in production systems (Sections 3–5)
3. **Augmentation taxonomy** classifying protective extension types (Section 6)
4. **Substrate analysis** arguing for UTXO/cell models as natural AMD substrates (Section 7)

---

2. Formal Framework

2.1 Definitions

Definition 1 (Mechanism). A mechanism $M = (S, A, f, P)$ consists of: - S: state space - A: action space (union of all agent action sets) - $f: S \times A \to S$: state transition function - $P: S \to R^n$: payoff function mapping states to agent utilities

Definition 2 (Core Property). A core property $\pi(M)$ is a predicate over mechanism outcomes that the designer considers essential. For a bonding curve, π might be "token price is a deterministic function of supply." For a Harberger tax, π might be "assets flow to highest-valued users."

Definition 3 (Vulnerability). A vulnerability $V(M)$ is a strategy profile σ^* available to a subset of agents such that: - σ^* is individually rational for those agents - The outcome under σ^* violates a normative goal G that motivated M's deployment - $G \neq \pi(M)$ — the vulnerability exploits the mechanism's social context, not its mathematical core

Definition 4 (Augmentation). An augmentation A is a transformation $A: M \to M'$ where: 1. **Core preservation**: $\pi(M') \Longrightarrow \pi(M)$ — the augmented mechanism preserves the core property 2. **Vulnerability**

mitigation: $V(M') \subset V(M)$ — the vulnerability set shrinks 3. **Bounded complexity**: $|M'| \leq c \cdot |M|$ for some constant c — augmentation adds bounded, not unbounded, complexity

Definition 5 (Augmentation Operator). Given mechanism M and vulnerability set $V = \{v_1, \ldots, v_k\}$, the augmentation operator produces:

```
A(M, V) = Aₖ(Aₖ₋₁(...A₁(M, v₁)..., vₖ₋₁), vₖ) = M'
```

Each A_i addresses vulnerability v_i independently. The composition is order-independent if augmentations target orthogonal vulnerability classes (Theorem 1).

2.2 Theorem 1: Compositional Augmentation

Theorem. If augmentations A_1 and A_2 target disjoint subsets of the state space ($S_1 \cap S_2 = \emptyset$ where S_i is the state modified by A_i), then:

```
A₁(A₂(M)) ≡ A₂(A₁(M))
```

Proof sketch. Since A_1 and A_2 modify disjoint state variables, neither augmentation's preconditions or postconditions can be invalidated by the other. The composed state transition function $f' = f_2 \circ f_1 = f_1 \circ f_2$ by independence of the modified state subspaces. ■

Corollary. Augmentations can be designed, tested, and deployed independently when they target orthogonal vulnerabilities — enabling modular security.

2.3 The Augmentation Spectrum

Not all mechanism modifications are augmentations. We distinguish:

Modification	Core Property	Complexity	Classification
Parameter tuning	Preserved	O(1)	Calibration

Modification	Core Property	Complexity	Classification
Protective extension	Preserved	O(k)	**Augmentation**
Mechanism hybridization	Partially preserved	$O(n^2)$	Synthesis
Mechanism replacement	Not preserved	O(n)	Redesign

AMD occupies the middle ground: more powerful than parameter tuning, less disruptive than replacement. The designer's judgment lies in identifying which failures warrant augmentation versus redesign.

---

3. Case Study 1: Augmented Bonding Curves (ABC)

3.1 The Pure Mechanism

A bonding curve (Hertzog, Benartzi & Benartzi, 2017) defines a deterministic price-supply relationship for a token. Given reserve R, supply S, and invariant function V:

```
V(R, S) = S^κ / R = V₀     (conservation law)
P = κR/S                     (spot price, derived)
```

The mechanism guarantees: tokens are always backed by reserves, price is a deterministic function of supply, and any agent can mint or burn at the curve price. This is the core property π.

3.2 The Vulnerabilities

V_1: Pump-and-dump. Early participants buy cheaply, attract later buyers, then dump their entire position. The curve faithfully executes the sell, crashing the price. The math is correct; the social outcome is predatory.

V_2: Front-running. Observing a pending large buy, an attacker buys first, captures the price movement, then sells. The curve cannot distinguish honest from adversarial ordering.

V_3: Misaligned incentives. Nothing binds early buyers to the project's success. Pure financial speculation dominates governance participation.

3.3 The Augmentation: ABC

The Augmented Bonding Curve (Titcomb, 2019; Zargham, Shorish & Paruch, 2020) adds four protective layers:

A_1: Dual-pool architecture. Split reserves into a Reserve Pool (backing token value, bonded to curve) and a Funding Pool (floating, allocated by governance). Entry and exit tributes flow from Reserve to Funding, creating a commons treasury that grows with activity.

A_2: Exit tributes. Sellers pay a friction fee φ on withdrawal:

```
Agent receives: (1 - ϕ) · ΔR
Funding Pool receives: ϕ · ΔR
```

This makes pump-and-dump strictly less profitable (attacker loses φ of gains) while funding the commons.

A_3: Hatch vesting. Early participants' tokens vest not by time but by governance participation — specifically, by allocating Funding Pool capital to projects. Vesting function:

```
S_vested = (1 - 2^(-γ(k - k₀))) · S_hatch
```

This binds early holders' liquidity to actual contribution, not speculation.

A_4: Front-running defense. Batched bonding curves (or, as VibeSwap implements, commit-reveal auctions) prevent observation of pending transactions. See Section 5.

3.4 Core Preservation Analysis

The conservation law V(R,S) = V0 is **unchanged**. Price remains a deterministic function of supply. Minting and burning still follow the invariant curve. The augmentations add structure *around* the curve without modifying the curve itself.

```
π(ABC) = π(BondingCurve) = "V(R,S) = V₀ for all state transitions"    ✓
V(ABC) ⊂ V(BondingCurve)   — pump-and-dump, front-running, misalignment
mitigated    ✓
```

3.5 Implementation

VibeSwap's `BondingCurveLauncher.sol` implements the augmented curve with: - Conservation invariant enforced per-transaction (256-bit wide multiplication via `mul_div`) - Exit tribute routed to `DAOTreasury.sol` Funding Pool - `CircuitBreaker.sol` for automated monitoring (Ostrom Principle 4) - `CommitRevealAuction.sol` for front-running defense (superior to batching — cryptographic commitment, not temporal batching)

---

4. Case Study 2: Augmented Harberger Taxes (AHT)

4.1 The Pure Mechanism

Harberger taxes (Posner & Weyl, 2017; Weyl & Zhang, 2018) implement a radical approach to property: owners self-assess the value of their asset, pay continuous tax proportional to that assessment, and anyone can force-purchase at the self-assessed price.

The mechanism creates a fundamental tension: - **High assessment** → high tax burden → owner pays for genuine valuation - **Low assessment** → low tax → anyone can force-buy cheaply

This tension is the core property π: **assets allocate to highest-valued users** because undervaluation is punished by cheap acquisition and overvaluation is punished by excessive tax.

4.2 The Vulnerability: Plutocratic Takeover

The pure mechanism's critical vulnerability is the **domain baron problem**: a well-capitalized actor can systematically force-acquire assets from legitimate holders who simply cannot afford to self-assess defensively.

Consider a user Alice who registers "alice.vibe" at a self-assessed value of 1 ETH (paying 0.05 ETH/year tax). A whale Bob can force-buy at 1 ETH — even though Alice's subjective value far exceeds 1 ETH — because Alice cannot afford to self-assess at 10 ETH (0.5 ETH/year tax) just to deter Bob.

The mechanism is correct: Alice *should* assess at her true value. But the mechanism ignores that Alice's true value may be primarily non-financial (identity, reputation, community standing) and that continuous tax on non-financial value is economically irrational.

This is the gap between mechanism theory and mechanism reality.

4.3 The Augmentation: AHT

The `.vibe network` (VibeNames.sol) implements five protective extensions:

A1: Loyalty Multiplier. The effective force-buy price increases with tenure × active usage:

```
function loyaltyMultiplier(uint256 tokenId) public view returns
(uint256) {
    VibeName storage v = _names[tokenId];
    if (!v.resolverActive) return BPS;  // Squatters get 1x forever
    uint256 tenure = block.timestamp - v.registeredAt;
    if (tenure >= LOYALTY_YEAR_5)  return 30_000;  // 3x
    if (tenure >= LOYALTY_YEAR_3)  return 20_000;  // 2x
    if (tenure >= LOYALTY_YEAR_2)  return 15_000;  // 1.5x
    return BPS;                                     // 1x (year 1)
}
```

Key design choice: loyalty only accrues if `resolverActive == true`. A name that resolves to nothing (squatter behavior) earns zero loyalty regardless of tenure. This separates genuine ownership from speculative hoarding at the mechanism level.

A2: Grace Period. Force-acquisition is not instant. A 72-hour right of first refusal gives the current owner time to respond:

```
initiateAcquire(tokenId, newValue) → escrow buyer's payment
  ↓ [72 hours]
owner can blockAcquisition(tokenId, newValue) → raise assessment, refund
buyer
  OR
completeAcquire(tokenId) → transfer to buyer after grace expires
```

This transforms a unilateral seizure into a negotiation. The owner can match or exceed the buyer's offer — converting the interaction from adversarial to price-discovery.

A3: Progressive Portfolio Tax. The tax rate scales superlinearly with portfolio size:

Names Owned	Tax Multiplier	Effective Annual Rate
1	1.0x	5.0%
2	1.5x	7.5%
3–5	2.0x	10.0%
6+	3.0x	15.0%

```
function portfolioTaxMultiplier(address owner) public view returns
(uint256) {
    uint256 count = portfolioSize[owner];
    if (count >= 6)  return PORTFOLIO_MULT_6_PLUS;   // 3x
    if (count >= 3)  return PORTFOLIO_MULT_3_5;       // 2x
    if (count >= 2)  return PORTFOLIO_MULT_2;         // 1.5x
    return PORTFOLIO_MULT_1;                           // 1x
}
```

A domain baron holding 10 names pays 3x the base rate *per name* — making hoarding economically irrational without affecting single-name owners.

A4: Acquisition Premium. The buyer pays an additional 20% premium (`ACQUISITION_PREMIUM_BPS = 2000`) on the effective force-buy price. This premium is paid directly to the displaced owner as compensation:

```
effectivePrice = selfAssessedValue × loyaltyMultiplier / BPS
totalCost = effectivePrice + (effectivePrice × ACQUISITION_PREMIUM_BPS /
BPS)
ownerCompensation = effectivePrice + acquisitionPremium
```

The premium ensures that even successful force-acquisition compensates the displaced owner above their self-assessed value — a structural acknowledgment that displacement has costs beyond the financial.

A5: Reputation Shield (Implicit). Because loyalty only accrues with active resolver usage, and resolver activity is publicly verifiable on-chain, a natural reputation layer emerges: long-tenured, actively-resolving names signal legitimate ownership without requiring explicit reputation oracles.

4.4 Core Preservation Analysis

The fundamental Harberger tension is **preserved**: self-assessment still determines both tax liability and acquisition vulnerability. A squatter with an unreasonably low assessment is still trivially force-bought. A hoarder with many names is taxed superlinearly.

What changes is the *gradient* of the tension. Pure Harberger is binary: too expensive or too cheap. AHT creates a smooth landscape where legitimate owners have a defensible position proportional to their genuine engagement.

```
π(AHT) = π(HarbergerTax) = "Assets flow to highest-valued users"    ✓
V(AHT) ⊂ V(HarbergerTax)   — plutocratic takeover, flash-acquisition
```

```
mitigated    ✓
```

4.5 Economic Invariants

The AHT system satisfies three invariants provable at the contract level:

1. **Tax monotonicity**: `∀t, tax(v₁) ≥ tax(v₂) ⟺ v₁ ≥ v₂` — higher assessment always means higher tax
2. **Loyalty monotonicity**: `∀t₁ > t₂, loyalty(t₁) ≥ loyalty(t₂)` — longer tenure never decreases defense
3. **Portfolio superlinearity**: `cost(n+1 names) > cost(n names) + cost(1 name)` — marginal cost of hoarding increases

4.6 Implementation

`VibeNames.sol` (878 lines) — ERC-721 + UUPS upgradeable + Augmented Harberger Tax, deployed with: - Per-second continuous tax streaming (no discrete collection points) - Dutch auction for expired names (7-day linear decay to 0) - Subdomain support (parent-authorized child names) - Full ERC-721 compatibility (names are tradeable NFTs)

---

5. Case Study 3: Commit-Reveal Batch Auctions (CRBA)

5.1 The Pure Mechanism

A continuous double auction (CDA) — the mechanism underlying most DEXes and all traditional exchanges — matches orders continuously as they arrive. The core property π: orders execute at the best available price with minimal latency.

5.2 The Vulnerability: MEV

Maximal Extractable Value (Daian et al., 2019; Flashbots, 2020) exploits the CDA's transparency. Because pending orders are visible in the mempool before execution, adversarial actors can:

- **Front-run**: insert an order before a large trade, capturing the price impact
- **Sandwich**: surround a trade with a buy-before and sell-after, extracting value from the user
- **Back-run**: execute immediately after a large trade to capture remaining price movement

MEV extraction on Ethereum exceeded $1.38B by 2023 (Flashbots MEV-Explore). This is not a bug in the CDA — it is a *consequence* of the CDA's design: transparent, sequential order processing.

5.3 The Augmentation: CRBA

VibeSwap's `CommitRevealAuction.sol` augments continuous trading with three temporal and cryptographic layers:

A_1: Temporal Decoupling (Commit Phase, 8s). Orders are submitted as cryptographic commitments during an 8-second commit window:

```
commitment = keccak256(abi.encodePacked(order, secret, address))
```

No information about order direction, size, or price is visible. The mempool contains only opaque hashes. Front-running requires breaking keccak256 — computationally infeasible.

A_2: Atomic Reveal (Reveal Phase, 2s). After the commit window closes, a 2-second reveal window opens. Users reveal their orders by providing the original plaintext and secret. Unrevealed commitments are slashed (50% of deposit forfeited to treasury).

The slashing mechanism converts "strategic non-reveal" (committing to probe the market, then not revealing) from a profitable strategy to a costly one.

A_3: Deterministic Fair Ordering. Revealed orders are shuffled using a Fisher-Yates shuffle seeded by XOR of all revealed secrets:

```
seed = secret₁ ⊕ secret₂ ⊕ ... ⊕ secretₙ
order = FisherYates(revealedOrders, seed)
```

No single participant can control the execution order. Manipulation requires controlling *all* revealed secrets — a colluding majority, which is visible on-chain and punishable.

A4: Uniform Clearing Price. All orders in a batch execute at the same clearing price, determined by supply-demand intersection within the batch. This eliminates price discrimination between orders in the same batch.

5.4 Core Preservation Analysis

Orders still execute at the best available price (now the uniform clearing price, which is mathematically provable as the efficient price given the batch's order set). Latency increases from milliseconds to ~10 seconds — a deliberate trade-off that converts latency advantage from an exploitable edge to a non-factor.

```
π(CRBA) ≈ π(CDA) = "Orders execute at efficient price"    ✓ (batch-
efficient, not point-efficient)
V(CRBA) ⊂ V(CDA)   — front-running, sandwich, information leakage
eliminated    ✓
```

5.5 Implementation

`CommitRevealAuction.sol` — UUPS upgradeable, 600+ lines, with: - `DeterministicShuffle.sol` library (Fisher-Yates with cryptographic seed) - `ProofOfWorkLib.sol` for optional priority bidding (willingness-to-pay for order priority) - `CircuitBreaker.sol` integration for volume/price/withdrawal anomaly detection - `PoolComplianceConfig.sol` for per-pool KYC/accreditation rules - Flash loan protection via EOA-only commits (contracts cannot submit commitments)

---

6. Augmentation Taxonomy

Across the three case studies, we identify five classes of augmentation:

6.1 Temporal Augmentations

Mechanism: Introduce time delays or windows between actions that are otherwise instantaneous.

Instance	Mechanism	Delay	Purpose
Commit window	CRBA	8 seconds	Decouple order submission from execution
Grace period	AHT	72 hours	Right of first refusal on acquisition
Hatch vesting	ABC	Half-life decay	Bind early holders to participation
Dutch auction	AHT	7 days	Gradual price discovery for expired names

Temporal augmentations convert instantaneous interactions into deliberative processes. They are effective when the vulnerability arises from *speed advantage* — the attacker profits by acting faster than the defender can respond.

6.2 Cryptographic Augmentations

Mechanism: Use cryptographic primitives to hide information until it is safe to reveal.

Instance	Mechanism	Primitive	Purpose
Commitment	CRBA	keccak256 hash	Hide order details during

Instance	Mechanism	Primitive	Purpose
			commit phase
XOR seed	CRBA	Secret XOR	Decentralized random seed generation
Reveal slashing	CRBA	Deposit forfeit	Punish strategic non-revelation

Cryptographic augmentations are effective when the vulnerability arises from *information asymmetry* — the attacker profits by observing the defender's intentions before acting.

6.3 Accumulative Augmentations

Mechanism: Reward or penalize based on cumulative behavior over time, not single interactions.

Instance	Mechanism	Accumulation	Purpose
Loyalty multiplier	AHT	Tenure × usage	Reward genuine long-term ownership
Conviction voting	ABC	Time-weighted stake	Reward consistent governance participation
Reputation oracle	CRBA	Behavioral history	Gate access based on track record

Accumulative augmentations are effective when the vulnerability arises from *ahistorical interactions* — the mechanism treats a first-time whale identically to a decade-long community member.

6.4 Progressive Augmentations

Mechanism: Scale costs, penalties, or barriers superlinearly with scale of activity.

Instance	Mechanism	Progression	Purpose
Portfolio tax	AHT	1x → 1.5x → 2x → 3x	Punish name hoarding
Exit tribute	ABC	% of withdrawal	Tax speculation proportionally
Rate limiting	CRBA	Volume caps per user	Prevent market domination

Progressive augmentations are effective when the vulnerability arises from *scale advantage* — the attacker profits by concentrating resources that individual defenders cannot match.

6.5 Compensatory Augmentations

Mechanism: Ensure that agents displaced by the mechanism receive explicit compensation.

Instance	Mechanism	Compensation	Purpose
Acquisition premium	AHT	20% of effective price	Compensate displaced name owners
Slashing redistribution	CRBA	50% to treasury, 50% to pool	Convert penalties to public goods
Funding pool allocation	ABC	Exit tributes → commons	Speculation tax funds community

Compensatory augmentations are effective when the vulnerability arises from *externalized costs* — the mechanism's operation imposes costs on agents who are not party to the transaction.

---

7. Substrate Analysis: UTXO/Cell Models as Natural AMD Substrates

7.1 Why Substrate Matters

Augmented mechanisms require explicit state management: loyalty must be tracked per-asset, portfolio size per-owner, commitments per-batch, vesting per-holder. The computational substrate determines how naturally these state requirements map to on-chain primitives.

7.2 Account Model Limitations

Ethereum's account model stores state in contract storage slots addressed by key. This creates three friction points for AMD:

1. **State coupling**: All augmentation state (loyalty, portfolio, pending acquisitions) lives in the same contract's storage. Upgrading one augmentation risks all others.
2. **Iteration cost**: Computing portfolio tax requires iterating over an owner's holdings — O(n) per operation in the worst case.
3. **Atomic composition**: Composing augmentations across contracts requires multi-call patterns or proxy delegation, adding gas and complexity.

7.3 CKB Cell Model Advantages

Nervos CKB's cell model (Jan, 2018) represents state as **cells**: independent, owned data units with explicit lock scripts (who can modify), type scripts (what modifications are valid), and data (the state itself).

This maps naturally to augmented mechanisms:

Loyalty as cell state. Each name registration creates a cell whose data includes `registeredAt` and `resolverActive`. The loyalty multiplier is computed directly from cell state — no contract storage iteration required.

Portfolio tax as cell counting. An owner's portfolio size is the count of cells they own with the VibeNames type script. CKB's indexer makes this O(1) via UTXO set queries, versus O(n) storage reads on Ethereum.

Grace period as cell lifecycle. A pending acquisition creates a new cell with a timelock script: the cell can only be consumed (completing the acquisition) after the grace period expires. The temporal augmentation is enforced at the cell level, not the contract level.

Composable augmentations. Each augmentation can be a separate type script. The loyalty type script validates loyalty calculations; the portfolio type script validates portfolio constraints; the grace period lock script enforces temporal delays. These compose naturally through CKB's transaction model — consuming input cells and producing output cells, with each script validating its own domain.

7.4 Formal Correspondence

AMD Concept	Ethereum Implementation	CKB Implementation
Mechanism state	Contract storage slots	Cell data fields
Augmentation logic	Modifier functions / require()	Type scripts (composable)
Temporal constraints	block.timestamp checks	Since (timelock in lock script)
Progressive scaling	Storage mapping iteration	UTXO set indexed query
Compositional safety	Multi-call / delegatecall	Transaction-level cell composition

The CKB cell model does not merely accommodate augmented mechanisms — it **encourages** them. Each augmentation is an independent verifiable script, composed at the transaction level, with explicit state ownership. This is AMD's formal structure made concrete.

---

8. The Augmentation Meta-Pattern

8.1 When to Augment vs. Redesign

Augmentation is appropriate when: 1. The core property π is desirable and well-understood 2. The vulnerability V is a consequence of context, not mathematics 3. Protective extensions can address V without modifying the core state transition f

Redesign is appropriate when: 1. The core property π is itself the source of harm 2. No bounded set of augmentations can address V 3. The mechanism's assumptions are fundamentally incompatible with the deployment context

8.2 Augmentation Anti-Patterns

Over-augmentation. Adding protective layers until the mechanism's core incentive is unrecognizable. If a Harberger tax has so many protections that force-acquisition is practically impossible, it has become a standard property regime with extra steps.

Asymmetric augmentation. Protecting one class of agents (e.g., incumbent owners) while leaving another class (e.g., new entrants) fully exposed. Good augmentations protect legitimate behavior regardless of agent identity.

Augmentation-as-governance. Using augmentation parameters (tax rates, grace periods, multiplier thresholds) as governance levers creates a meta-vulnerability: the augmentation itself becomes a political tool. Prefer fixed, transparent, mathematically justified parameters.

8.3 Relationship to Cooperative Capitalism

Augmented Mechanism Design is the formal foundation of what VibeSwap calls **Cooperative Capitalism**: the synthesis of free-market competition (the pure mechanism) with mutualized protection (the augmentation layer).

- Pure bonding curves are free-market capitalism: buy low, sell high, no safety net.
- ABC is cooperative capitalism: the market still works, but exit tributes fund the commons and vesting binds early capital to participation.
- Pure Harberger taxes are radical market efficiency: assets flow to highest bidder.
- AHT is cooperative capitalism: assets still flow efficiently, but loyalty rewards genuine ownership and portfolio tax prevents concentration.
- Pure continuous auctions are open competition: fastest wins.
- CRBA is cooperative capitalism: competition still determines price, but cryptographic commitment eliminates information advantage.

The pattern is universal: **preserve the competitive core, augment the cooperative shell.**

---

9. Related Work

- **Harberger, 1962**: Original self-assessed property taxation proposal
- **Posner & Weyl, 2017**: *Radical Markets* — modern revival of Harberger taxes, COST proposal
- **Weyl & Zhang, 2018**: Formal treatment of partial common ownership
- **Titcomb, 2019**: Augmented Bonding Curves for commons funding (Commons Stack)

- **Zargham, Shorish & Paruch, 2020**: Configuration spaces formalization of bonding curves (BlockScience/WU Vienna)
- **Daian et al., 2019**: *Flash Boys 2.0* — formalization of MEV on Ethereum
- **Budish, Cramton & Shim, 2015**: Frequent batch auctions as alternative to continuous trading
- **Ostrom, 1990**: *Governing the Commons* — 8 principles for commons management
- **Hurwicz, 1960**: Optimality and informational efficiency in resource allocation mechanisms
- **Myerson, 1981**: Optimal auction design
- **Jan, 2018**: Nervos CKB cell model and generalized UTXO architecture

---

10. Conclusion

Pure economic mechanisms are not failures — they are *foundations*. Their mathematical elegance captures real allocative principles. Their social vulnerabilities are consequences of deployment context, not design error.

Augmented Mechanism Design provides a systematic methodology for building on these foundations: identify the core property, enumerate the contextual vulnerabilities, design orthogonal protective extensions, and verify that the augmented mechanism preserves the core while mitigating the threats.

The three case studies — ABC, AHT, and CRBA — demonstrate that this pattern is not ad hoc but structural. The same augmentation classes (temporal, cryptographic, accumulative, progressive, compensatory) appear across radically different mechanism domains. This universality suggests that AMD is a fundamental design methodology, not a collection of tricks.

We propose that the field of mechanism design adopt augmentation as a first-class design primitive, alongside the traditional tools of incentive compatibility, revelation principles, and implementation theory. And we propose that the CKB/Nervos cell model — with its explicit state ownership, composable verification scripts, and natural support for temporal constraints — provides the most architecturally coherent substrate for deploying augmented mechanisms on-chain.

The future of decentralized systems is not pure mechanisms or protected mechanisms. It is **augmented mechanisms**: mathematically rigorous, socially aware, and cooperative by construction.

Appendix A: VibeSwap Contract Addresses (Base Mainnet)

All case study mechanisms are deployed or deployment-ready. Source code: github.com/wglynn/vibeswap

Appendix B: Augmentation Constant Reference

Constant	Value	Mechanism	Purp
TAX_RATE_BPS	500 (5%/yr)	AHT	Base tax o asses
GRACE_PERIOD	72 hours	AHT	Righ refus wind
ACQUISITION_PREMIUM_BPS	2,000 (20%)	AHT	Com to dis owne
PORTFOLIO_MULT_6_PLUS	30,000 (3x)	AHT	Maxi portf multi

Constant	Value	Mechanism	Purp
LOYALTY_YEAR_5	1,825 days	AHT	Maxi loyal defer
COMMIT_PHASE	8 seconds	CRBA	Orde comr wind
REVEAL_PHASE	2 seconds	CRBA	Orde revel wind
SLASH_RATE	50%	CRBA	Pena unrev comr
EXIT_TRIBUTE	φ (configurable)	ABC	Specu tax o witho
DUTCH_AUCTION_DURATION	7 days	AHT	Price perio expir name

"Fairness Above All." — P-000, The Lawson Constant, VibeSwap Protocol